Language Models and Inference

CS 541 Lecture 6

Zining Zhu



STEVENS
INSTITUTE OF TECHNOLOGY
1870

From n-grams to neural language models

The next-word game · a short history

The language modeling objective

A **language model** assigns a probability to sequences of tokens, and is usually trained to predict the next token given the ones before it.

- Formally: model $P(\text{next token} \mid \text{all previous tokens})$, written

$$P(x_t \mid x_1, x_2, \ldots, x_{t-1})$$

- Training signal is everywhere: any piece of text is a free source of “next token” labels
- The same objective scales from toy corpora to internet-scale pretraining

Predicting the next word turns out to require a surprising amount of world knowledge.

Why next-token prediction is a free, universal signal

- No human annotator is needed: every sentence ever written already contains its own labels
  - "The cat sat on the ___" — the label is whatever word actually came next
- Contrast with supervised learning (Lecture 2): there, someone had to hand-label each example
- This is called **self-supervised learning**: the supervision is carved out of the data itself
- Because raw text is abundant, the **training set size** stopped being the bottleneck it is for labeled data

A toy example: counting next words

Suppose our entire training corpus is these three short sentences:

- “the cat sat”, “the cat ran”, “the dog sat”
- Count how often each word follows “the cat”: “sat” once, “ran” once → $P(\text{sat} \mid \text{the cat}) = P(\text{ran} \mid \text{the cat}) = 0.5$
- Count how often each word follows “the”: “cat” twice, “dog” once → $P(\text{cat} \mid \text{the}) = 2/3$, $P(\text{dog} \mid \text{the}) = 1/3$
- This **is** a language model — just an extremely small, sparse one built from raw counts

Real corpora repeat this idea at a trillion-token scale, with a learned neural network in place of a counting table.

n-gram models

- Approximate $P(x_t \mid \text{everything before it})$ using only the last $n-1$ tokens
- Counts from a corpus give probabilities directly: no gradient descent needed
- **Sparsity**: most n-grams never appear in any corpus, however large
- **No long-range context**: a 3-gram model forgets everything more than 2 tokens back
- Smoothing (Laplace, Kneser-Ney) patches sparsity but not the context limit

Smoothing: patching the zero-count problem

- Raw counting assigns probability **zero** to any n-gram never seen in training — a single unseen phrase can crash a whole sequence's probability to zero
- **Laplace (add-one) smoothing**: pretend every possible n-gram occurred at least once
  - Simple, but wastes a lot of probability mass on n-grams that are genuinely impossible
- **Kneser-Ney smoothing**: backs off to lower-order n-grams weighted by how many **distinct** contexts a word appears in, not just raw frequency
  - Captures that a word appearing in many different contexts is more likely to generalize to a new one
- Neither smoothing method solves the deeper problem: fixed, short context windows

Try it yourself: estimate a trigram probability

Given these trigram counts from a corpus:

Trigram	Count
“not very good”	3
“not very bad”	1
“not very much”	0

- Using add-one (Laplace) smoothing over a vocabulary of 1,000 possible third words, what is $P(\text{good} \mid \text{not very})$?
- What happens to $P(\text{much} \mid \text{not very})$ — the unseen case — under the same smoothing?

Work it out, then check the next slide.

Solution: the trigram walkthrough

- Add-one smoothing:

$$P(w \mid \text{not very}) = \frac{\text{count}(w) + 1}{\text{total count} + \text{vocabulary size}}$$

- Total count after "not very" $= 3 + 1 + 0 = 4$; vocabulary size $= 1{,}000$
- $P(\text{good} \mid \text{not very}) = (3 + 1)/(4 + 1000) = 4/1004 \approx 0.0040$
- $P(\text{much} \mid \text{not very}) = (0 + 1)/(4 + 1000) = 1/1004 \approx 0.0010$
- The unseen word "much" gets a small but **nonzero** probability — that is exactly what smoothing buys us

Perplexity: scoring a language model

- **Perplexity** measures how "surprised" a model is by held-out text: lower is better
- Intuition: perplexity ≈ the effective number of equally-likely next-token choices the model is choosing among
- A model that always assigns the true next token probability 1 has perplexity 1 (perfect); a uniform random guesser over a vocabulary of size V has perplexity V
- Perplexity is computed from the model's own probability estimates on text it did not train on, never on text it has memorized

This is the n-gram/RNN/Transformer era's shared scoreboard — everything in this lecture is trying to drive perplexity down.

Check your understanding

- Why is next-token prediction called **self-supervised**, and why does that matter for scale?
- What specifically does an n-gram model forget that a longer-context model would remember?
- What problem does smoothing fix, and what does perplexity measure?



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Recurrent neural networks

A running memory, one token at a time

From fixed windows to a running memory

- An n-gram model sees only the last $n - 1$ tokens; everything earlier is forgotten by construction
- A **recurrent neural network** (RNN) keeps a **hidden state** h_t: a vector that summarizes everything read so far
- At each step it reads one token, updates the hidden state, and predicts the next token
- The same weights are reused at every step, so one model handles sequences of any length

The RNN update rule

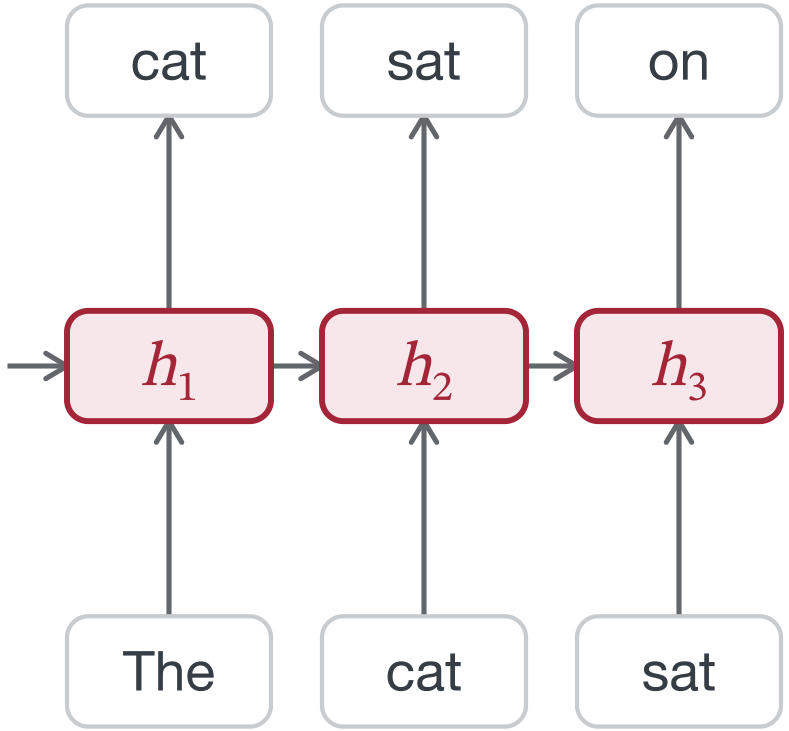
$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

$$P(x_{t+1} \mid x_1, \ldots, x_t) = \text{softmax}(W_o h_t)$$

- x_t: the vector for the current token; h_{t-1}: the memory from the previous step, with $h_0 = 0$
- W_h, W_x, W_o are shared across all steps
- The hidden state is the **only** channel through which the past reaches the present

Unrolling an RNN through time

- Draw one copy of the cell per token: the weights repeat, only the state changes
- Information flows forward along the h arrows
- Each h_t predicts the next token; the loss adds up next-token cross-entropy over all steps



Example: a memory that fades

One-number hidden state, $h_t = \tanh(0.5\, h_{t-1} + x_t)$, $h_0 = 0$. Feed $x_1 = 1$, then $x_2 = x_3 = 0$:

Step	Input	Computation	h_t
1	1	$\tanh(0.5 \cdot 0 + 1)$	**0.762**
2	0	$\tanh(0.5 \cdot 0.762)$	**0.363**
3	0	$\tanh(0.5 \cdot 0.363)$	**0.180**

- The signal from step 1 roughly halves at every step; by step 10 it is about 0.001
- The model has effectively forgotten the first token

Training an RNN: backpropagation through time

- Unroll the network over the whole sequence, compute the loss at every step, then backpropagate through the unrolled graph
- The gradient reaching step 1 passes through every W_h and every $\tanh$ on the way back: a product of many factors
- Factors below 1 make the gradient **vanish**; factors above 1 make it **explode**
- Gradient clipping tames explosions; vanishing needs a change of architecture

Why RNNs are slow and forget

- **Sequential bottleneck**: to compute the hidden state at position 100, you must first compute positions 1 through 99, one at a time — training cannot parallelize across the sequence
- **Vanishing gradients**: when backpropagating through many time steps, gradient signal shrinks (or explodes) multiplicatively at each step
  - A dependency 50 tokens back can become numerically invisible to the optimizer
- LSTMs and GRUs add gates to fight this, but the sequential-compute problem remains architectural, not just numerical

Gated RNNs: LSTM and GRU

- **Gates** are learned values between 0 and 1 that decide what to keep, write, and expose
- The **LSTM** adds a separate cell state updated additively, a near-direct path for information and gradients across many steps
- The **GRU** is a lighter variant with fewer gates and parameters
- The workhorse of 2010s translation and speech, but still sequential: step t must wait for step $t - 1$

2017: what “Attention Is All You Need” changed

- Replaced the recurrent hidden state with **self-attention**: every token computes its representation by looking directly at every other token, in one parallel step
- Removed the sequential dependency at training time entirely — a full batch of sequences trains as one large matrix multiplication
- Attention’s paths between any two positions are length 1 (direct), versus length = distance for an RNN — this is why long-range dependencies became far easier to learn
- Enabled the scale (billions of tokens, then trillions) that the rest of this lecture depends on

A short timeline, revisited

Era	Approach	Main limitation
1990s-2000s	n-gram counting	sparsity, fixed short context
2010s	RNN / LSTM	sequential training, vanishing gradients
2017 onward	Transformer	quadratic attention cost
2020s	Scaled Transformer pretraining	data/compute cost (Lecture 7)

Each era's "main limitation" becomes the next era's research problem — a pattern worth watching for in every AI subfield, echoing Lecture 1's history of AI milestones.

Check your understanding

- Why can an RNN process sequences of any length with a fixed number of parameters?
- In the example, why did the step-1 signal shrink, and what happens to the gradient if the recurrent factor is above 1?
- In one sentence, what changed about **training** (not inference) when attention replaced recurrence?

If any of these feel shaky, flag it now — the Transformer builds directly on them.



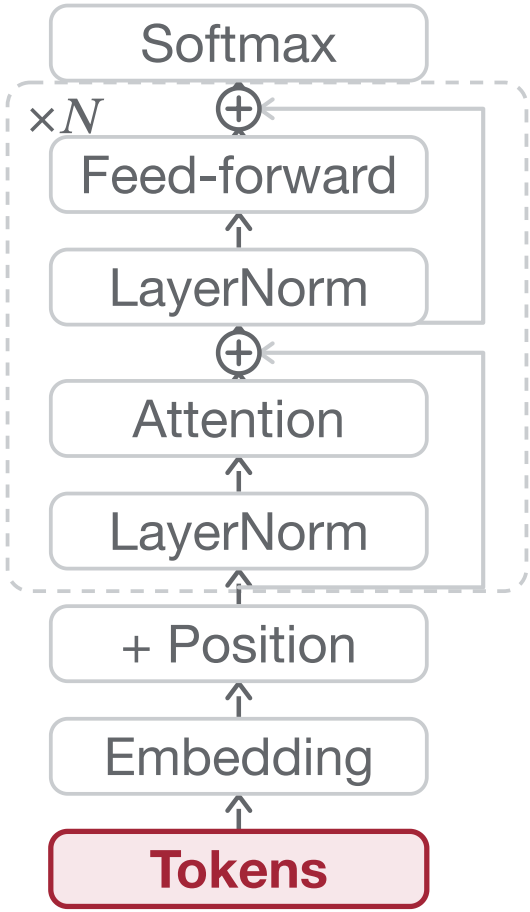
STEVENS
INSTITUTE OF TECHNOLOGY
1870

The Transformer architecture

Tokens in, tokens out

Tokenization

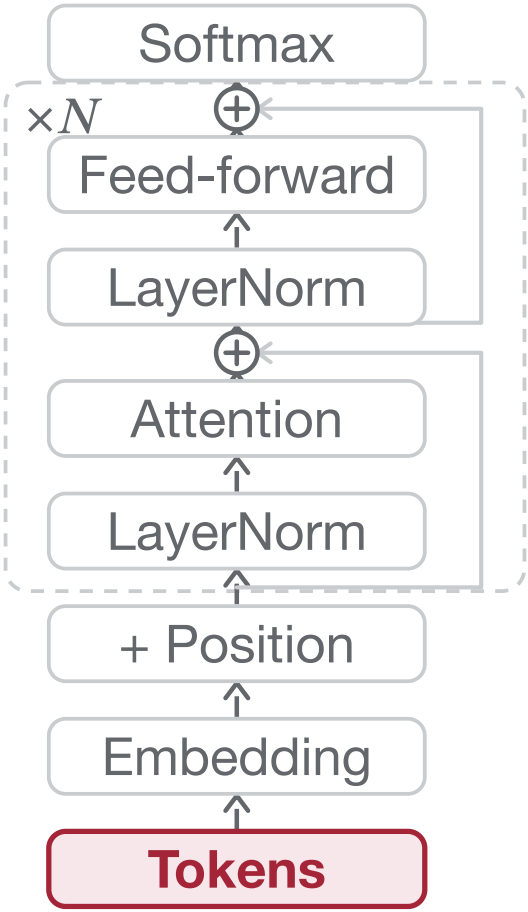
- Text is split into **tokens**: sub-word pieces, not full words or characters
  - Balances vocabulary size against sequence length
- Each token maps to an integer id, then to a learned embedding vector



Why sub-word tokens, not words or characters

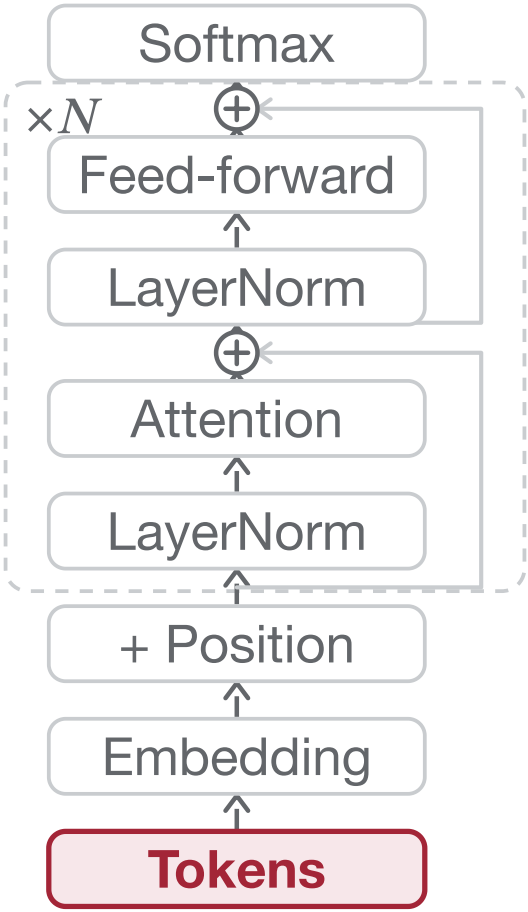
Granularity	Vocabulary size	Sequence length	Handles unseen words?
Whole word	Huge (open-ended)	Short	No — unseen word = unknown token
Character	Tiny (~100)	Very long	Yes, always
Sub-word	Moderate (30k-100k)	Moderate	Yes, degrades gracefully

Sub-word tokenization is the compromise that keeps both vocabulary size and sequence length manageable. Common words often stay whole; rare words split into meaningful pieces.



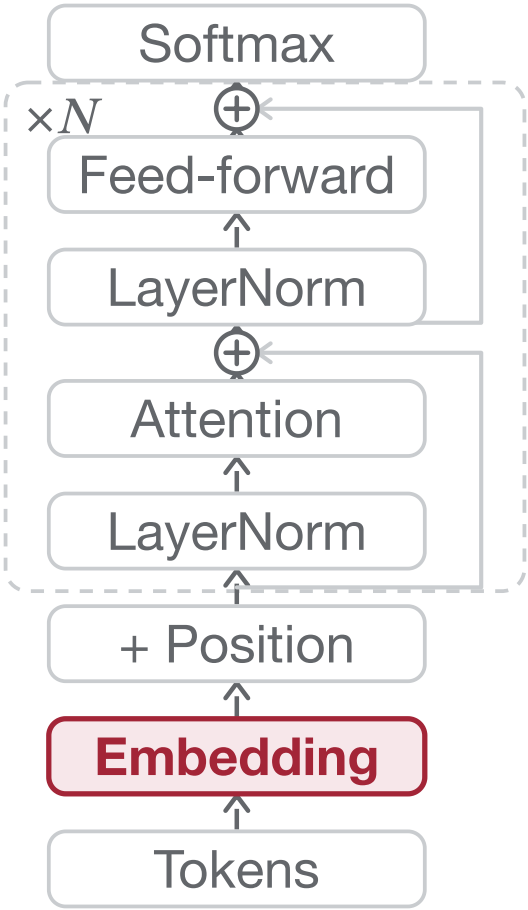
Tokenization edge cases

- **Numbers**: “3.14159” may split into several digit-groups, which is part of why arithmetic on many digits is hard for language models
- **Code**: indentation, symbols, and camelCase identifiers tokenize unevenly across languages and styles
- **Multilingual text**: languages underrepresented in the tokenizer’s training corpus fragment into many more tokens per word — this quietly makes those languages more expensive to process and generate
- Tokenizer choice is a design decision made once, up front, and is expensive to change later



Embeddings

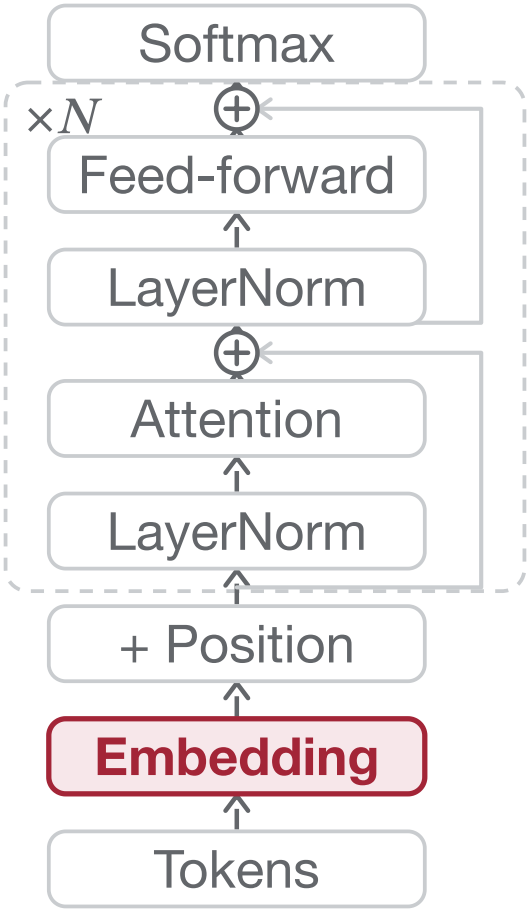
- An embedding table maps each token id to a dense vector in a d-dimensional space
- These vectors are learned, not designed: similar-usage tokens end up nearby
- The embedding is the model's **input representation**; a similar (often tied) matrix projects back to vocabulary logits at the output



What embedding geometry captures

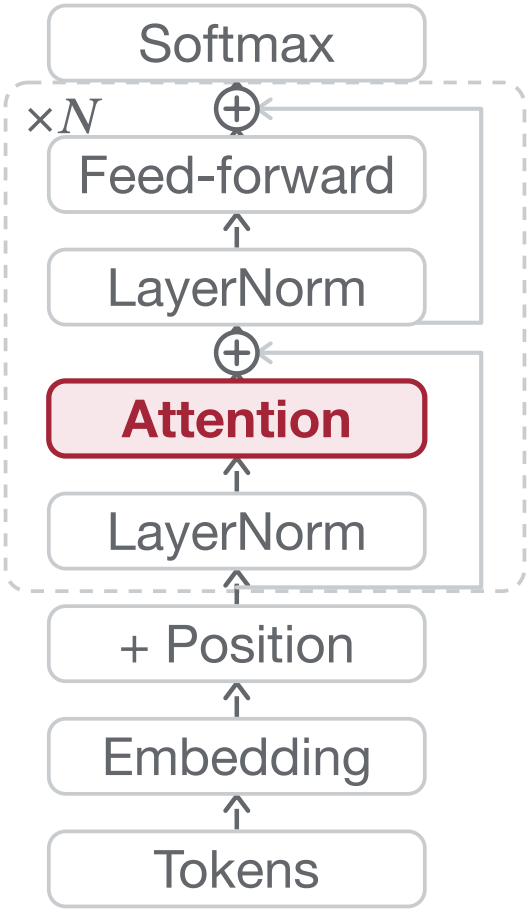
- Tokens used in similar contexts are pulled toward similar vectors purely by the next-token prediction objective — no one tells the model “these are synonyms”
- A widely cited (and somewhat idealized) illustration: $\mathrm{vec}(\mathrm{king}) - \mathrm{vec}(\mathrm{man}) + \mathrm{vec}(\mathrm{woman})$ lands close to $\mathrm{vec}(\mathrm{queen})$
  - Treat this as a simplified illustration of what training **can** encourage, not a guaranteed property of every embedding space
- This geometric structure is a side effect of the training objective, not something explicitly programmed in

Nothing here is symbolic reasoning about royalty — it is statistics over co-occurrence patterns that happens to look structured.



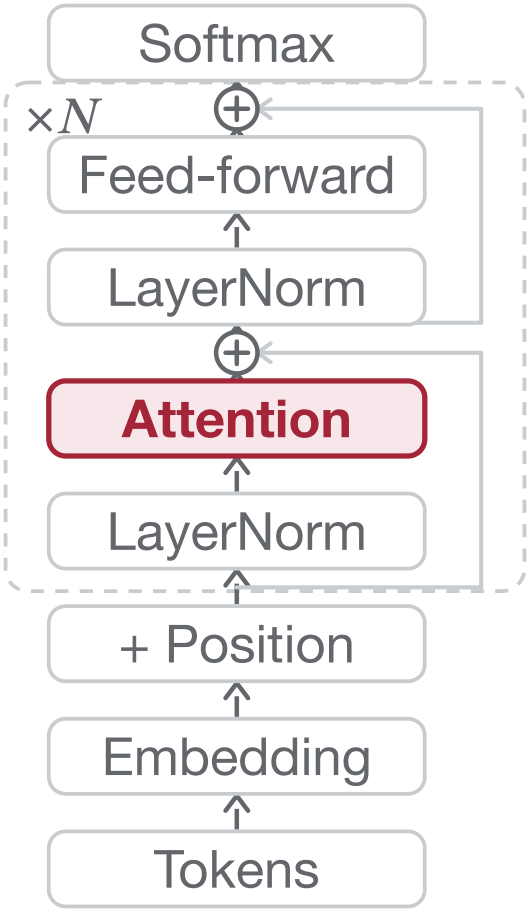
Self-attention: the intuition

- Each token produces three vectors: a **query**, a **key**, a **value**
- A token's output is a weighted average of **all** tokens' values
  - Weights come from how well its query matches each token's key
- This lets a token pull in exactly the context it needs, regardless of distance



Query, key, value: a library-search analogy

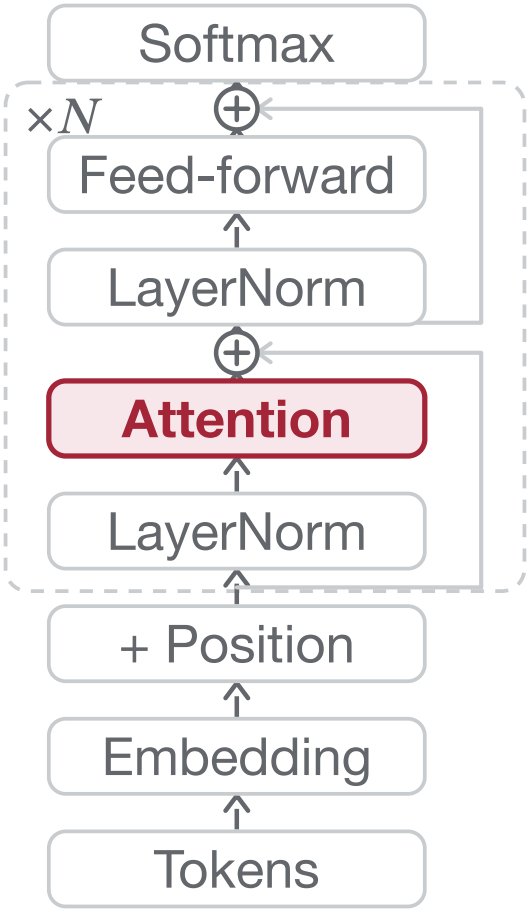
- Think of every token as holding an index card in a library
- Its **key** is like the card's subject tags — what this card is "about"
- Its **value** is the actual content on the card — what it contributes if selected
- Its **query** is the search request another token sends out: "what am I looking for right now?"
- Attention compares one token's query against every other token's key, and returns a blend of values weighted by how well each key matches



Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- $QK^\top$: similarity scores between every pair of positions (each query dotted with each key)
- Divide by $\sqrt{d_k}$ (d_k is the dimension of each key vector) to keep gradients well-scaled as dimension grows
- Softmax turns scores into a probability distribution over positions to attend to
- In a language model, a **causal mask** blocks attention to future tokens

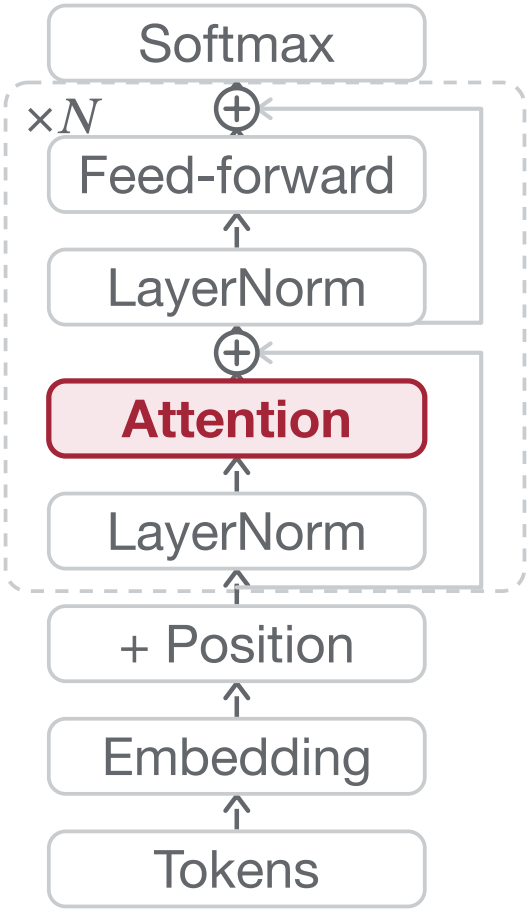


Self-attention example: setup

A 3-token toy sequence: “The”, “cat”, “sat”. Suppose each key/value vector has dimension 2, and these are the (hand-chosen, for illustration) key and value vectors:

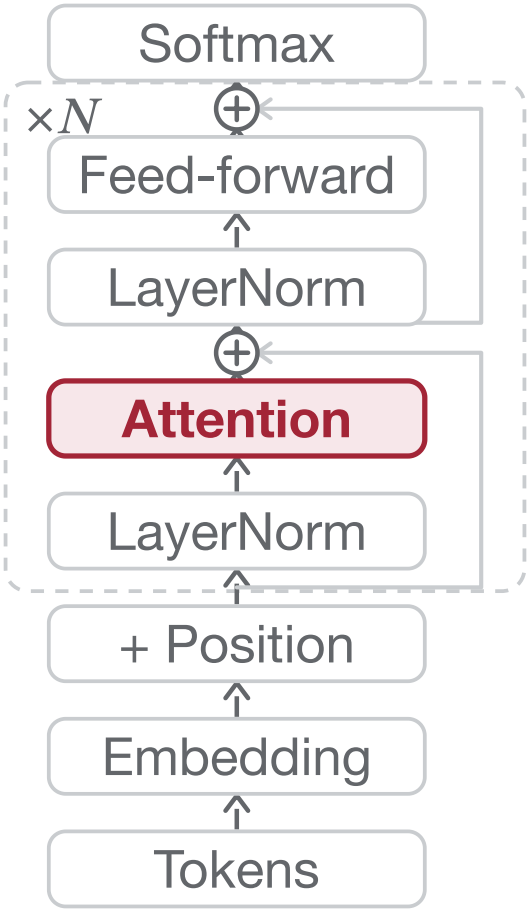
Token	Key	Value
The	$(1, 0)$	$(1, 2)$
cat	$(0, 1)$	$(3, 1)$
sat	$(1, 1)$	$(0, 4)$

- Query for “cat”: $q = (1, 0)$
- We will compute the attention output for the token “cat”



Self-attention example: the scores

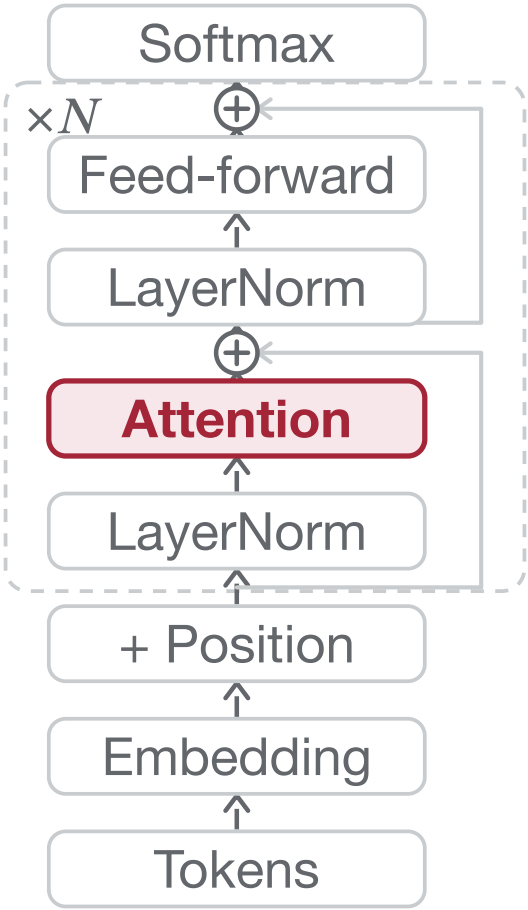
- Score against “The”: $q \cdot k = (1)(1) + (0)(0) = 1$
- Score against “cat”: $q \cdot k = (1)(0) + (0)(1) = 0$
- Score against “sat”: $q \cdot k = (1)(1) + (0)(1) = 1$
- Scale by $\sqrt{d_k} = \sqrt{2} \approx 1.414$: scaled scores $\approx 0.707, 0, 0.707$



Self-attention example: softmax and output

- $\mathrm{softmax}(0.707, 0, 0.707) \approx (0.401, 0.198, 0.401)$ – "The" and "sat" get equal, larger weight; "cat" attends less to itself here
- Weighted sum of values: $0.401 \cdot (1, 2) + 0.198 \cdot (3, 1) + 0.401 \cdot (0, 4) \approx (1.0, 2.6)$
- That vector, $(1.0, 2.6)$, is the new, context-aware representation for "cat" at this layer

Every token repeats this whole computation, in parallel, using its own query.

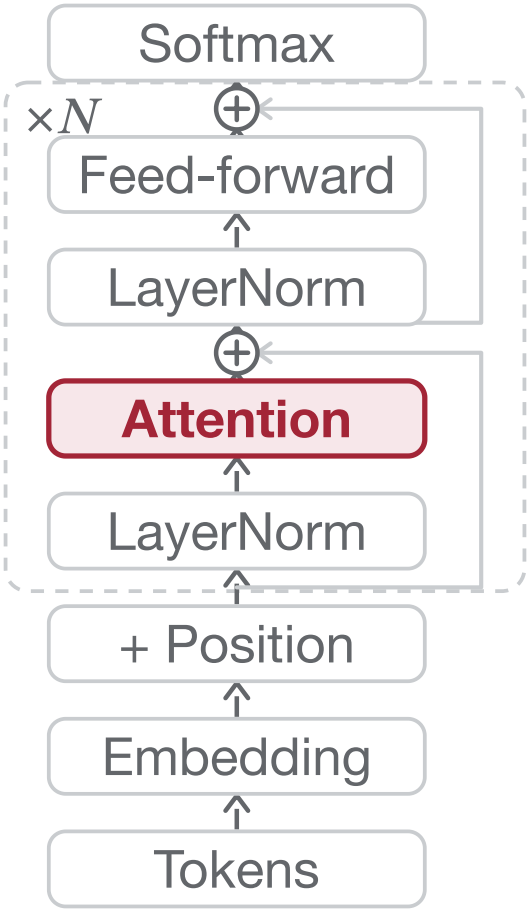


Try it yourself: attention for “sat”

Using the same keys and values as before, and query for “sat” $q = (0, 1)$:

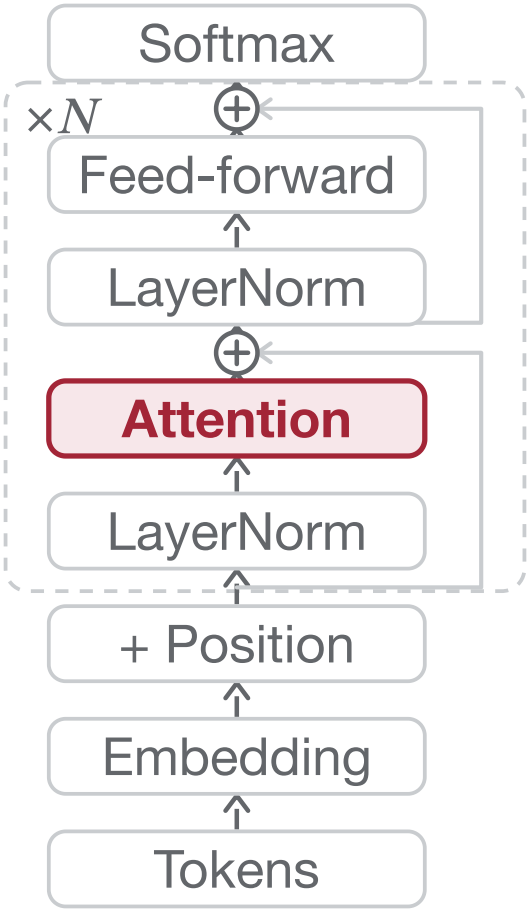
- Compute the three raw scores $q \cdot k$ against “The”, “cat”, “sat”
- Scale by $\sqrt{d_k} = \sqrt{2}$, apply softmax, and compute the weighted sum of values

Work it out, then check the next slide.



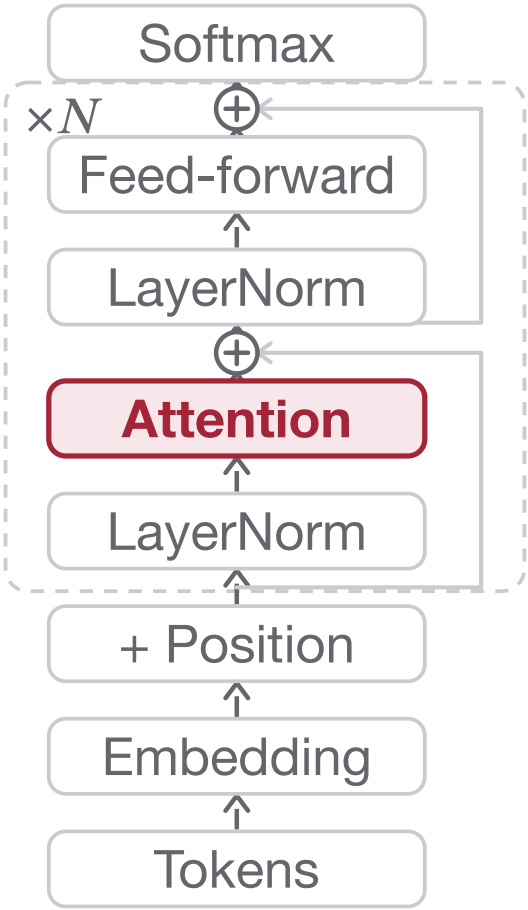
Solution: attention for “sat”

- Scores: $q \cdot k_{\text{The}} = 0, q \cdot k_{\text{cat}} = 1, q \cdot k_{\text{sat}} = 1$
- Scaled: $0, 0.707, 0.707$
- $\text{softmax}(0, 0.707, 0.707) \approx (0.198, 0.401, 0.401)$
- Weighted output: $0.198 \cdot (1, 2) + 0.401 \cdot (3, 1) + 0.401 \cdot (0, 4) \approx (1.4, 2.4)$
- Notice “sat” ends up attending mostly to “cat” and itself, and barely to “The” — a plausible syntactic pattern (the subject matters more than the article for this verb)



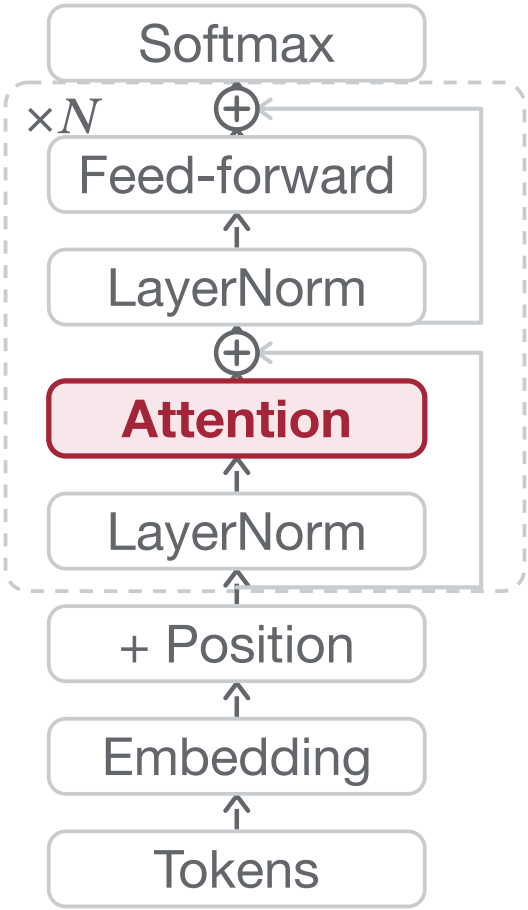
Why divide by $\sqrt{d_k}$

- Before scaling, dot products of high-dimensional vectors tend to have large magnitude, growing with d_k
- Large logits going into softmax produce a nearly one-hot distribution — gradients through softmax vanish almost everywhere
- Dividing by $\sqrt{d_k}$ keeps the scores in a range where softmax gradients stay informative, regardless of how large d_k is
- This single design choice is why attention trains stably at the dimensions used in practice (hundreds per head)



The causal mask for language modeling

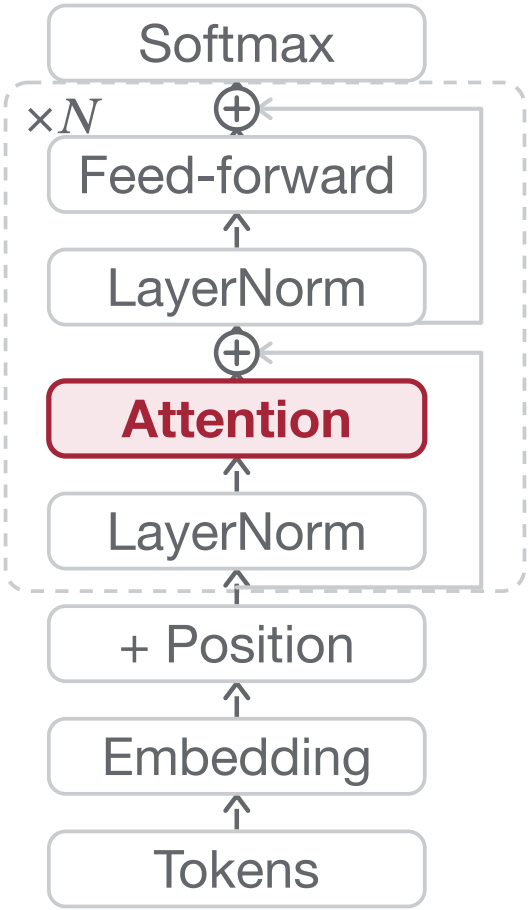
- A language model must not “peek” at future tokens when predicting the current one
- Before softmax, scores from position t to any position after t are set to $-\infty$
- Softmax then assigns those positions exactly zero weight — token t’s representation only ever mixes information from positions $\leq t$
- This is what makes the same architecture usable for both training (see the whole sequence, but masked) and generation (only the past exists yet)



Multi-head attention

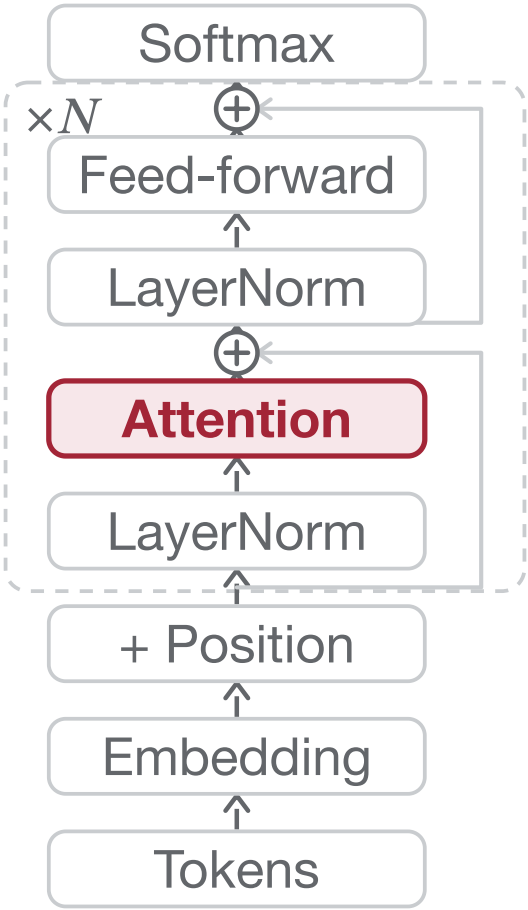
- Run several attention operations in parallel, each with its own Q, K, V projections
- Each **head** can specialize: one might track syntax, another coreference, another position
- Outputs of all heads are concatenated and linearly projected back to d dimensions

More heads is not automatically better — it's a tradeoff against per-head dimension.



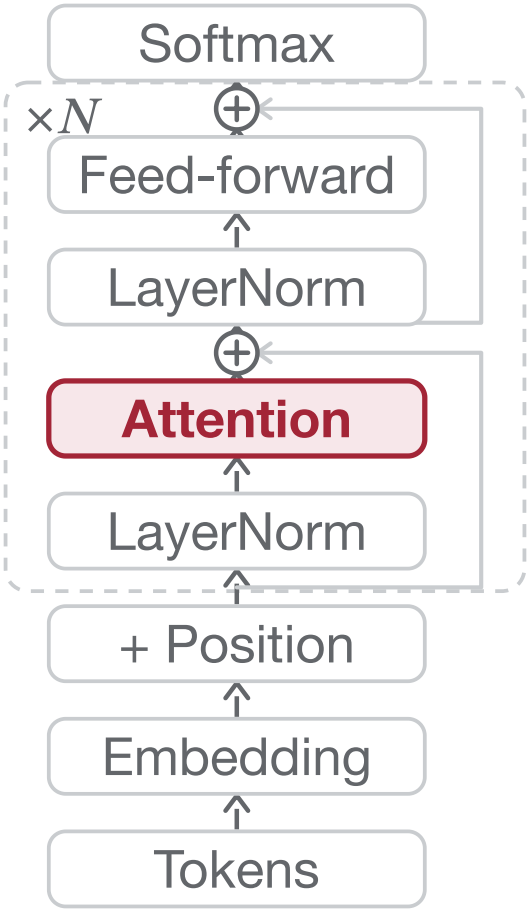
Why multiple heads help

- A single attention operation produces exactly one weighting pattern per token — but useful context often depends on several different relationships at once
- Example: resolving “it” in “The trophy didn’t fit in the suitcase because it was too big” needs one signal for grammatical role and another for plausible physical size
- With multiple heads, one head’s pattern can specialize toward nearby words (local syntax), another toward the sentence’s subject (long-range reference), without either being forced to share a single weighting



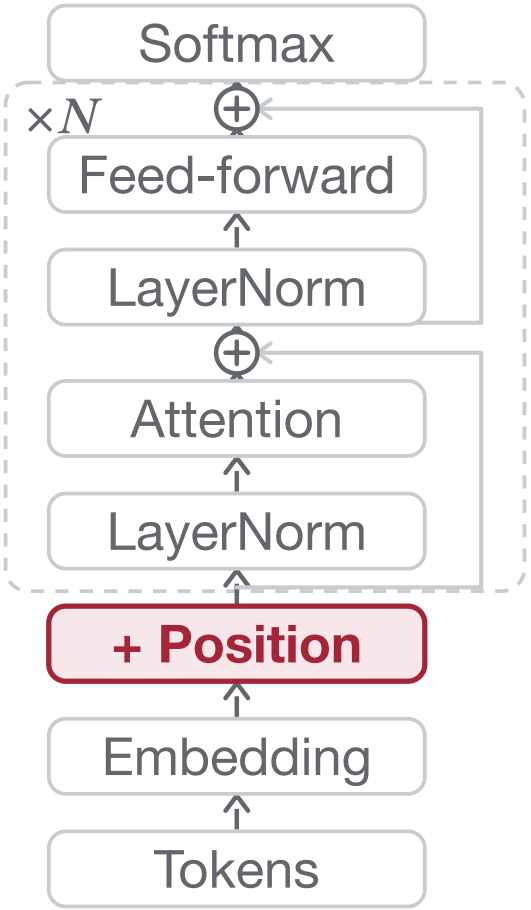
Multi-head arithmetic

- Suppose the model dimension is $d = 512$ and we use 8 heads
- Each head works in a smaller space: $d_k = d/\text{heads} = 512/8 = 64$ dimensions
- Every head computes its own attention independently, in that 64-dimensional space
- The 8 outputs (each 64-dimensional) are concatenated back to 512 dimensions, then passed through one more learned projection



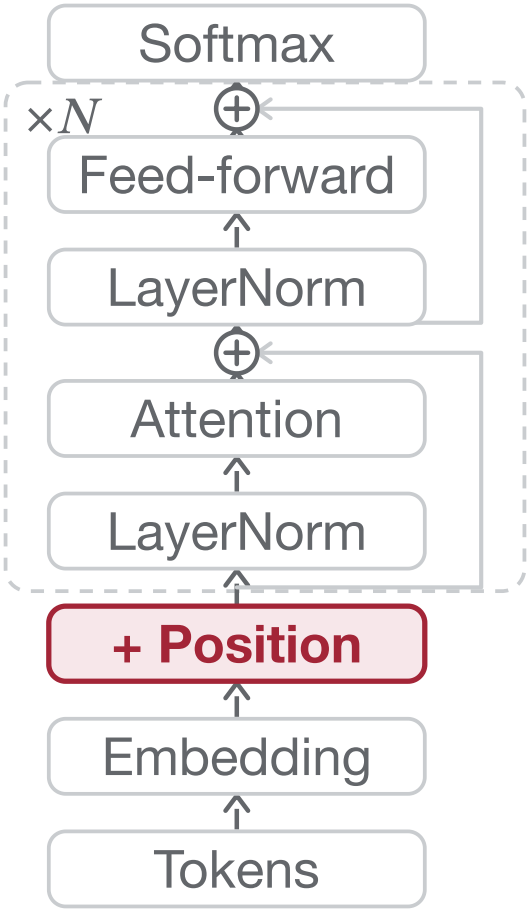
Positional encoding: why order matters

- Self-attention treats the input as a **set** of tokens: swap two positions and, without extra help, the attention pattern between them is unchanged
- Word order carries meaning ("dog bites man" $\neq$ "man bites dog") — the model needs an explicit signal for **where** each token sits
- **Positional encoding**: information about position is injected, usually by adding a position-dependent vector to each token's embedding



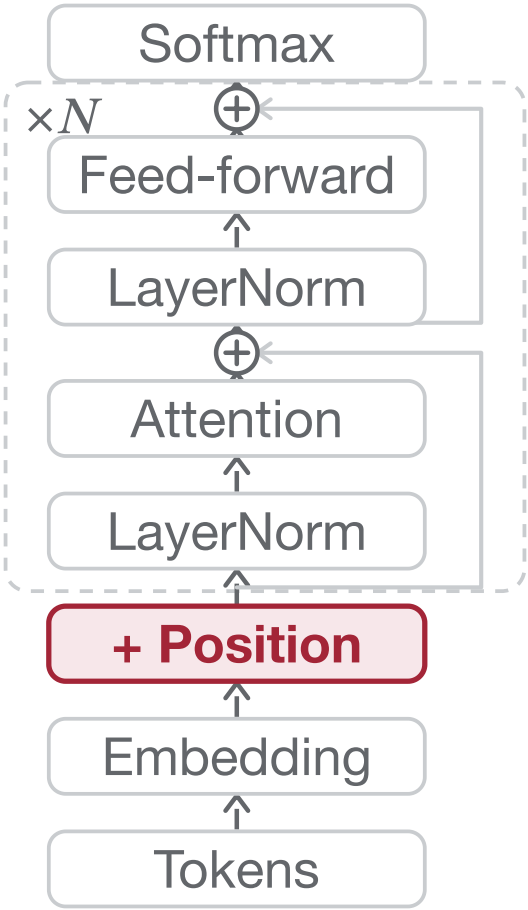
Sinusoidal positional encodings

- The original Transformer adds a fixed vector to each position, built from sine and cosine waves of different frequencies
- Each dimension of the vector oscillates at a different frequency, so every position gets a unique combined pattern
- Because the pattern is a smooth function of position, nearby positions get similar encodings, and the pattern generalizes to sequence lengths not seen in training
- These vectors are **fixed**, not learned — no extra parameters are needed



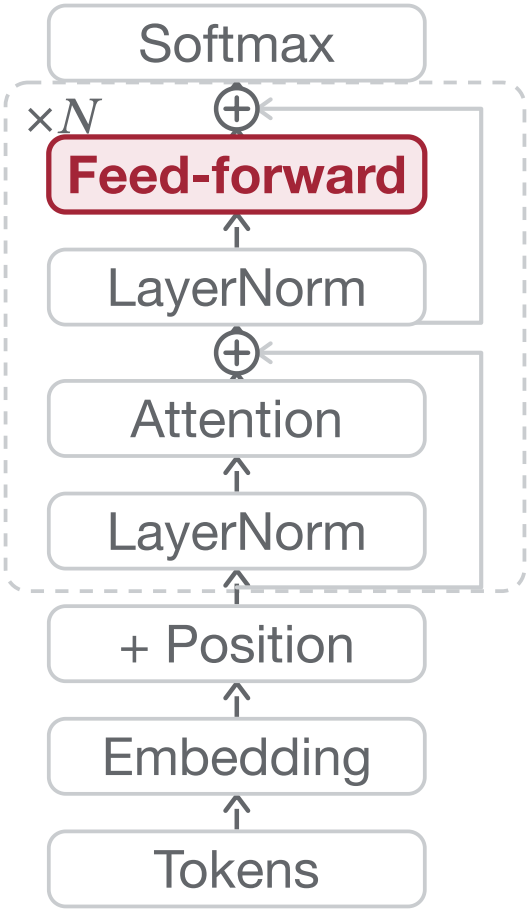
Modern positional alternatives

- **Rotary position embeddings (RoPE)**: instead of adding a position vector, **rotate** the query and key vectors by an angle that depends on position, before computing attention scores
  - The resulting dot product naturally depends on the **relative** distance between positions, not their absolute location
- **Relative position biases**: add a learned bias to the attention score, based only on how far apart two positions are
- Both are popular in recent large language models because relative position tends to generalize better to sequences longer than any seen in training



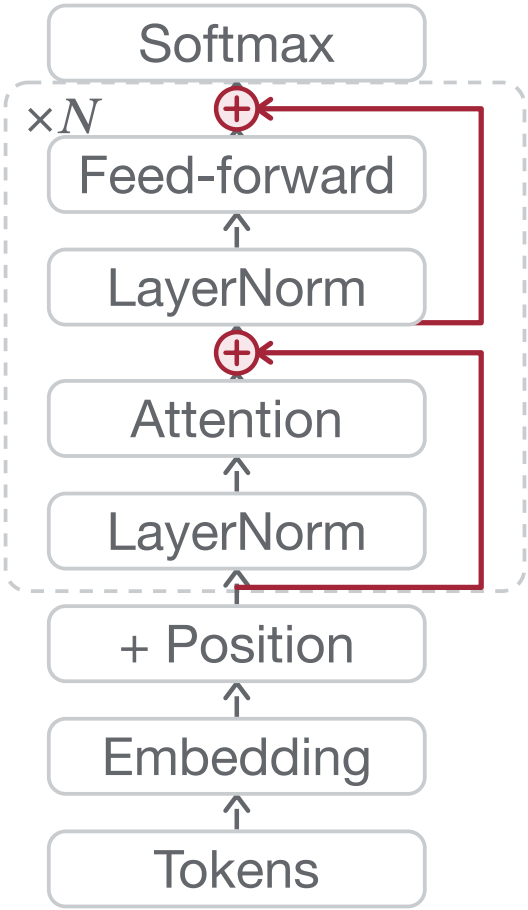
Feed-forward sublayer

- After attention mixes information **across** positions, a feed-forward sublayer processes each position **independently**
- Structure: a linear layer up to a wider hidden dimension, a nonlinearity, then a linear layer back down
- Typically the widest part of the network — the hidden dimension is often $4\times$ the model dimension
- Attention decides **what to look at**; the feed-forward layer decides **what to do with it**, per position



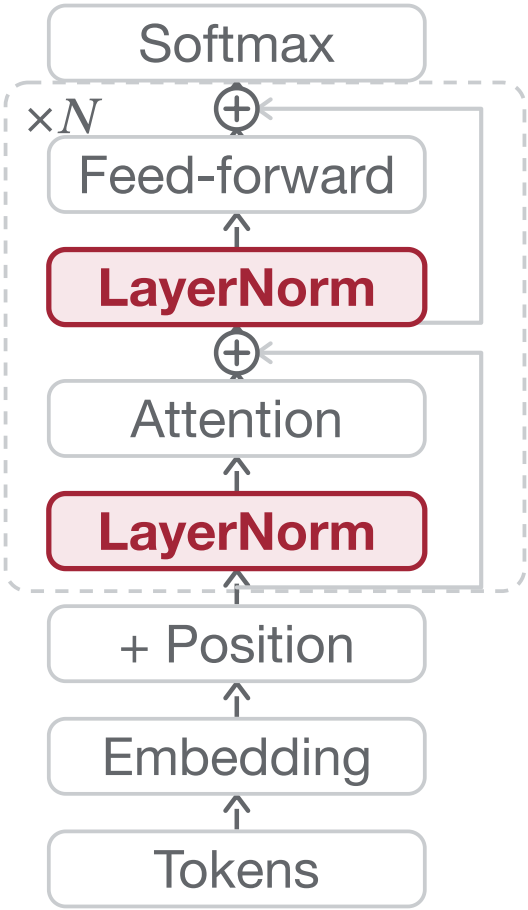
Residual connections

- Each sublayer's output is **added back** to its input, rather than replacing it: $\text{output} = \text{input} + \text{Sublayer}(\text{input})$
- Guarantees a direct path for gradients to flow from the output all the way back to the input, even through dozens of stacked layers
- Recall from Lecture 2: deep networks without this trick are hard to optimize because gradients can vanish across many layers — residual connections were one of the key fixes that made very deep networks trainable at all
- Without residuals, stacking as many Transformer blocks as modern models use would likely not train



Layer normalization

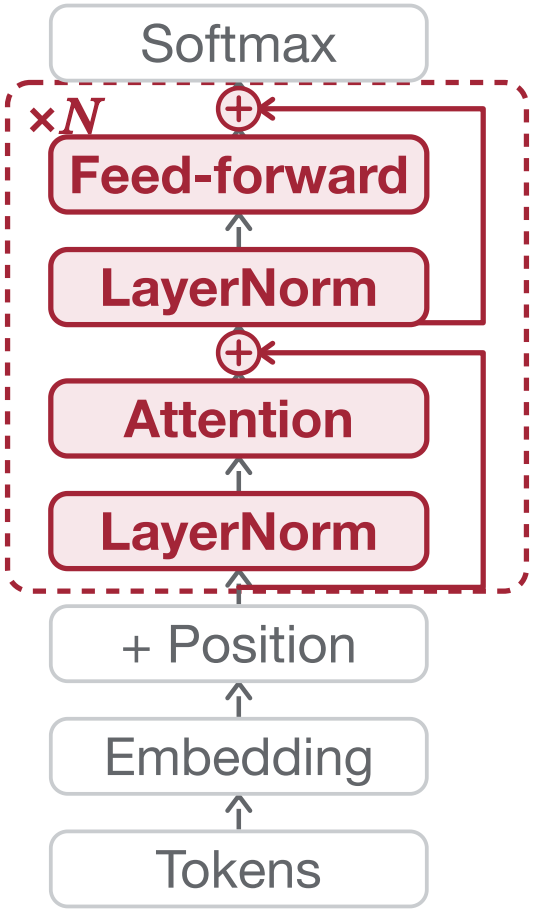
- Normalizes the activations at each position to have consistent scale (roughly zero mean, unit variance) before they enter a sublayer
- Stabilizes training by preventing activation magnitudes from drifting to extreme values as they pass through many layers
- Applied per-token, independent of the rest of the batch — unlike some earlier normalization schemes tied to batch statistics
- Paired with residual connections, this combination is what makes 50+ layer Transformers trainable in practice



Assembling a full Transformer block

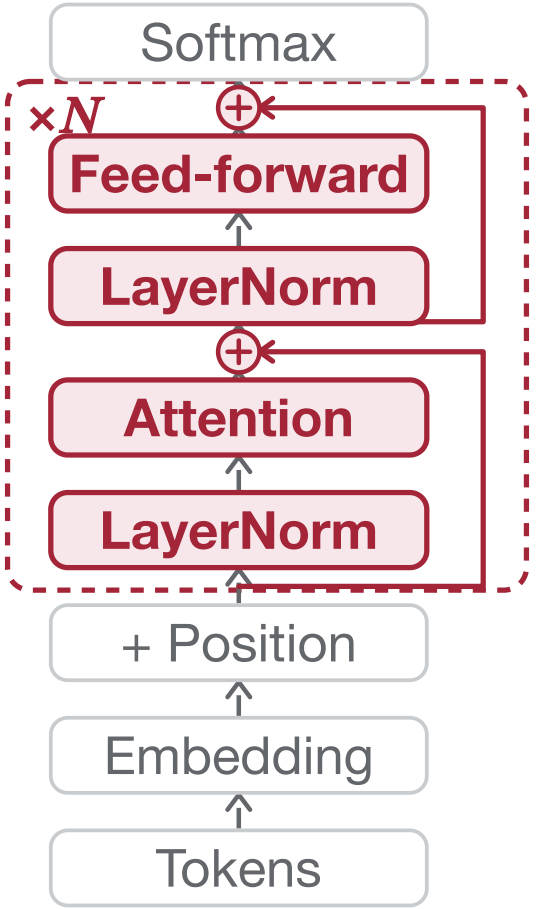
- One block, in order: layer norm → self-attention → add residual → layer norm → feed-forward → add residual
- Input and output of the block have the same shape, so blocks can be stacked arbitrarily deep
- Every block repeats the same two-step pattern: mix information across positions (attention), then process each position (feed-forward)

Stack this block N times (N in the tens for today's large models) to get the full network.



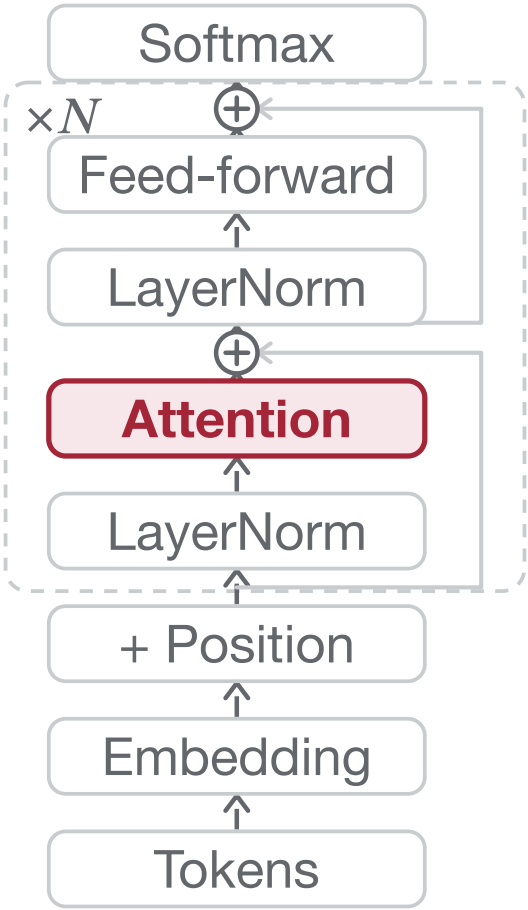
Stacking blocks: depth vs. width

- **Depth** (more blocks): more sequential rounds of “look around, then think” — helpful for multi-step reasoning-like patterns
- **Width** (larger model dimension, more heads): more capacity per round to represent rich features
- In practice, both are scaled together; going deep without going wide (or vice versa) tends to waste compute
- This tradeoff reappears, formally, in Lecture 15’s discussion of model capacity



Attention's cost, and efficient variants

- Computing $QK^\top$ compares every position against every other position: cost grows with the **square** of sequence length
- Doubling the context window roughly quadruples the attention compute — a real ceiling on how long a context window can be
- **Sliding-window attention**: each token only attends to a fixed-size local window, trading long-range reach for linear cost
- **Sparse / linear attention variants**: approximate full attention with structured patterns or low-rank tricks to reduce the quadratic cost
- We return to the practical consequences of this cost in Lecture 8's deployment discussion



Encoder-only, decoder-only, encoder-decoder

Architecture	Attention direction	Typical use
Encoder-only (BERT-style)	Bidirectional (sees full input)	Understanding: classification, embeddings
Decoder-only (GPT-style)	Causal (only sees the past)	Open-ended generation
Encoder-decoder (T5-style)	Encoder bidirectional, decoder causal	Sequence-to-sequence: translation, summarization

Today's large conversational language models are overwhelmingly decoder-only, because a single causal model can both understand and generate.

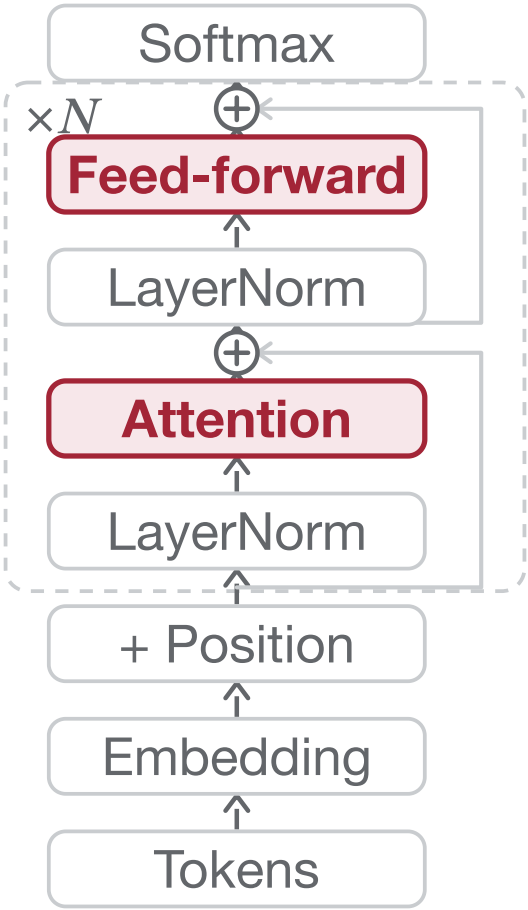
Masked LM vs. causal LM objectives

- **Causal (autoregressive) LM**: predict each token from only the tokens before it — the objective this whole lecture has been building toward
- **Masked LM**: hide a random subset of tokens in the input and predict them using context from **both** directions (used by BERT-style encoders)
- Masked LM produces excellent representations for understanding tasks, but does not naturally support left-to-right generation
- Causal LM is slightly less data-efficient per token, but generation "falls out" of the objective for free — part of why it became the dominant choice for general-purpose assistants

Parameter-count estimate

Take a toy Transformer with model dimension $d = 4096$ and $L = 32$ layers (roughly the scale of a small production-grade model, for illustration only).

- Attention projections (Q, K, V, and output): 4 matrices of size $d \times d \to 4d^2$ parameters per layer
- Feed-forward sublayer (up-projection and down-projection, $4\times$ expansion): 2 matrices of size $d \times 4d \to 8d^2$ parameters per layer
- Total per layer $\approx 12d^2$; across L layers $\approx 12 \times L \times d^2$
- Plugging in: $12 \times 32 \times 4096^2 \approx 6.4$ billion parameters (excluding the embedding table, which is comparatively small at this scale)



Check your understanding

- Why does attention need a scaling factor of $\sqrt{d_k}$, and what breaks without it?
- In your own words, what problem do residual connections solve, and where else in this course have you seen that problem before?
- Why is a decoder-only, causal architecture the natural fit for a general-purpose chat assistant?

This is the deepest section of the lecture — worth a second pass on your own notes.



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Inference

From weights to generated text

Autoregressive decoding

- The model produces one token at a time: sample x_t from $P(\cdot \mid x_1, \ldots, x_{t-1})$
- Each new token is appended to the input and fed back in to produce the next
- This is inherently **sequential**: token $t+1$ needs token t to already exist
- Contrast with training, which is parallel across all positions of a sequence at once

Training-time parallel, inference-time sequential

	Training	Inference (generation)
What's known	The entire target sequence, up front	Only the tokens generated so far
Compute pattern	One large parallel pass over the whole sequence	One token at a time, repeated
Bottleneck	Total FLOPs for the batch	Latency per step, times number of steps

This asymmetry is the root cause of most of the deployment engineering in Lecture 8.

Sampling strategies

- **Greedy decoding**: always pick the highest-probability token
  - Deterministic, but often repetitive and dull
- **Temperature**: rescale logits before softmax
  - Low temperature → sharper, more confident distribution; high → flatter, more random
- **Top-k sampling**: sample only from the k most likely tokens
- **Top-p / nucleus sampling**: sample from the smallest set of tokens whose probabilities sum to p

No single strategy is "correct" — the right choice depends on whether you want reliability or diversity.

Sampling example

A toy next-token distribution over 5 tokens:

Token	A	B	C	D	E
Probability	0.40	0.25	0.15	0.12	0.08

- **Greedy**: always outputs A
- **Top-k ($k = 2$)**: samples only from $\{\mathrm{A}, \mathrm{B}\}$, renormalized to $(0.615, 0.385)$
- **Top-p ($p = 0.9$)**: cumulative sum $\mathrm{A}(0.40) \to +\mathrm{B}(0.65) \to +\mathrm{C}(0.80) \to +\mathrm{D}(0.92)$ crosses 0.9, so the nucleus is $\{\mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{D}\}$; E (0.08) is excluded

Try it yourself: top-p sampling

A different toy distribution over 5 tokens:

Token	V	W	X	Y	Z
Probability	0.50	0.20	0.15	0.10	0.05

- With $p = 0.9$, which tokens make up the nucleus, and which are excluded?

Work it out, then check the next slide.

Solution: top-p sampling

- Cumulative sum: $\mathrm{V}(0.50) \rightarrow +\mathrm{W}(0.70) \rightarrow +\mathrm{X}(0.85) \rightarrow +\mathrm{Y}(0.95)$ — this crosses 0.9, so the nucleus is $\{\mathrm{V}, \mathrm{W}, \mathrm{X}, \mathrm{Y}\}$
- Z (0.05) is excluded
- Renormalizing over the nucleus (dividing by 0.95): $\mathrm{V} \approx 0.526$, $\mathrm{W} \approx 0.211$, $\mathrm{X} \approx 0.158$, $\mathrm{Y} \approx 0.105$
- Sampling then draws from just these four, according to the renormalized probabilities

Repetition and degeneration

- Greedy decoding, and low-temperature sampling generally, are prone to falling into repetitive loops (“I think that I think that I think that…”)
- One cause: once a phrase repeats, the model’s own training data rarely showed such a loop being broken, so the highest-probability continuation can be “repeat again”
- Sampling-based methods (temperature, top-k, top-p) inject enough randomness to escape these loops, at the cost of occasional less-coherent output
- Some serving systems add explicit repetition penalties on top of any of these strategies

Latency, throughput, and batching

- **Latency**: time to produce one response for one user
- **Throughput**: total tokens produced per second across all users
- Batching multiple requests together improves throughput, often at the cost of latency
- Serving systems must balance these under real traffic patterns
- A preview of Lecture 8's deployment topics: parallelism strategies exist precisely to manage this tradeoff at scale

Check your understanding

- Why is inference sequential when training over the same sequence was parallel?
- What does batching more requests together trade off, and why?
- Give one reason a system designer might choose top-p over greedy decoding for a creative-writing product, and one reason they might choose greedy for a different product.



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Wrap-up: from raw text to an assistant

What’s still missing · what’s next

What a language model system needs

Three independent design choices combine to produce the systems you interact with:

- **Architecture**: how tokens turn into contextualized representations (attention, positional encoding, the Transformer block)
- **Training objective and data**: what the model is taught to predict, and on what (next week: self-supervised pretraining at scale)
- **Decoding strategy**: how a trained model's probabilities turn into actual output text (greedy, temperature, top-k, top-p)

Hallucination, revisited

- The pretraining objective rewards producing **plausible** continuations of text, not **true** ones
- A model trained purely this way has no built-in mechanism to say “I don’t know” — fluent, confident-sounding, incorrect text is exactly what the objective would reward if it happens to look plausible
- This is not a bug that better architecture alone fixes — it is a direct consequence of what next-token prediction optimizes for
- Reducing hallucination is one of the central goals of the post-training methods covered next week

The “stochastic parrot” debate

- One position: large language models are sophisticated statistical pattern-matchers over text, with no genuine understanding — “stochastic parrots” recombining what they’ve seen
- Another position: at sufficient scale, the patterns a model must internalize to predict text well enough start to functionally resemble structured world knowledge and limited reasoning
- This is squarely the same question Lecture 1 opened with: what should count as “intelligence,” and does behavior alone settle the question?
- The debate remains open in the research community; forming your own informed opinion is one of the goals of taking this course

A pretrained model is not yet an assistant

Everything today used one objective: predict the next token on raw text.

- A base model trained this way is a powerful text completion engine, but:
  - It is not reliably helpful, harmless, or honest by default
  - It will complete a harmful prompt just as readily as a benign one
  - It has no notion of “answer the user’s question” versus “continue this document”

Closing that gap, from raw completion to aligned assistant, is the subject of the last part of next week’s lecture: **post-training**.

Looking ahead

- Next week (Lecture 7): how such a model is trained at scale and prompted, and how it becomes an instruction-following assistant — pretraining, in-context learning, supervised fine-tuning, RLHF (PPO), DPO, and RLVR/GRPO
- The architecture and inference machinery from today does not disappear — training and post-training reuse all of it, just with a different objective layered on top
- Assignment 3 (Search) is due today; start thinking ahead to how the search and learning threads of this course reconverge once you reach the reinforcement learning lectures



STEVENS
INSTITUTE OF TECHNOLOGY
1870

THANK **YOU**

Stevens Institute of Technology
1 Castle Point Terrace, Hoboken, NJ 07030